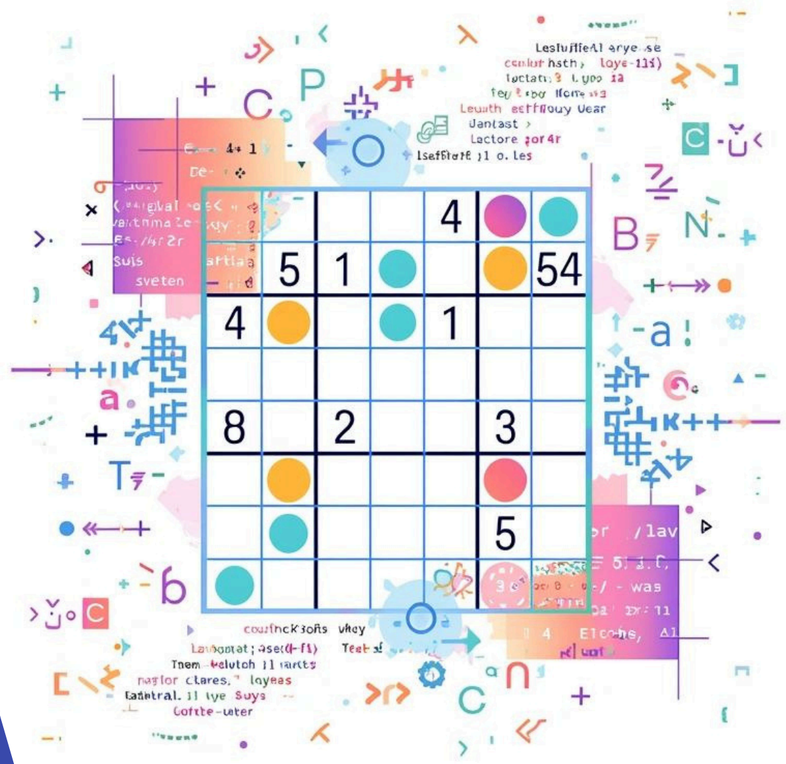
C Mini Project: 4×4 Sudoku Game

PRESENTED BY : ARYA CHALIKWAR : 1262251699
DEETYA PANCHAL : 1262251777
HELLY GANDHI : 1262250532
SHAMBHAVI KATYAYAN : 1262252522



Team Roles Overview

LOGIC DESIGN

Shambhavi was responsible for creating the overall structure, focusing on **developing algorithms** and ensuring adherence to Sudoku rules.

CODING IMPLEMENTATION

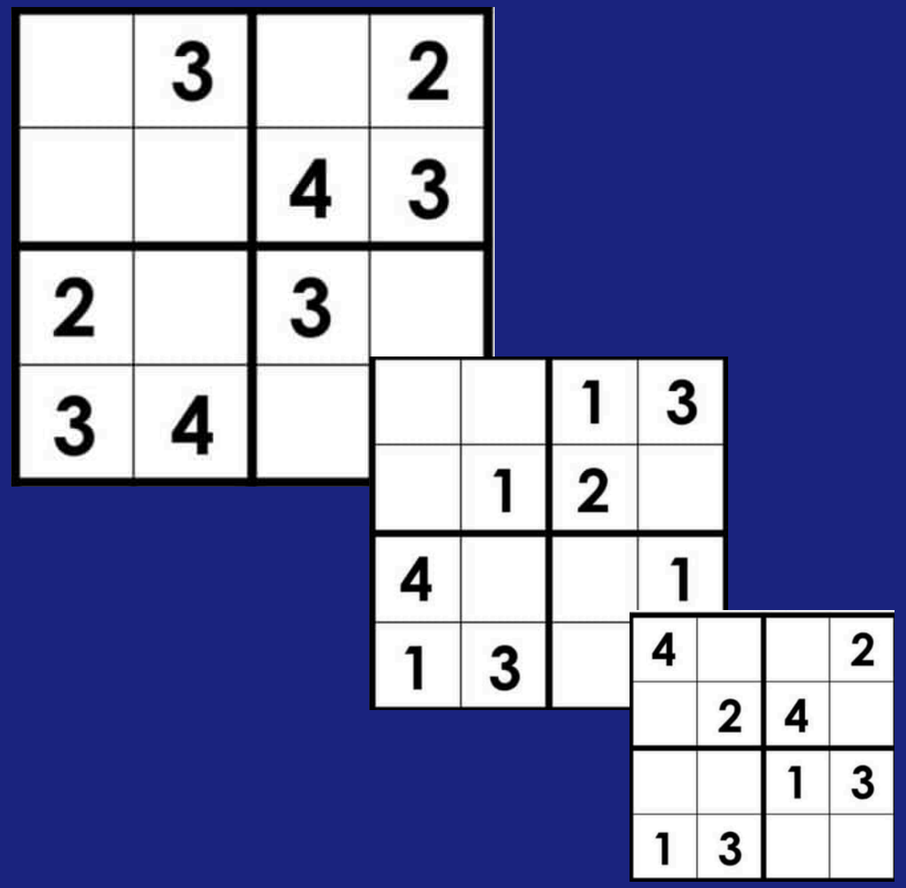
Helly and Arya took charge of writing the core C program, expertly implementing features **to streamline user interaction** and maintain efficiency.

TESTING PROCESS

Deetya conducted thorough testing, identifying bugs and enhancing performance, ensuring the final product was **reliable and user-friendly** for all players.

Key Features of Our Sudoku Game

- INTERACTIVE 4×4 SUDOKU GRID FOR USERS.
- AUTOMATICALLY GENERATES A NEW SOLVABLE SUDOKU PUZZLE EVERY TIME.
- DIFFERENT PUZZLE EACH TIME WHEN USER ENTERS -1.
- DETECTS WRONG NUMBERS AND SHOWS WARNING MESSAGES.
- DISPLAYS A CONGRATULATORY MESSAGE WHEN SUDOKU IS COMPLETED.
- ENSURES SUDOKU RULES (NO REPETITION IN ROWS AND COLUMNS).
- USER-FRIENDLY INPUT SYSTEM (ROW AND COLUMN FROM 1 TO 4).
- REPLAY OPTION WITHOUT RESTARTING THE PROGRAM.
- IMPLEMENTED USING CORE C PROGRAMMING CONCEPTS LIKE ARRAYS, LOOPS, CONDITIONS, AND FUNCTIONS.



Understanding How Our Sudoku Game Functions

GAME FLOW OVERVIEW

1. The program starts and generates a random solvable 4×4 Sudoku grid.
2. The Sudoku board is displayed with some cells empty.
3. The user enters the row, column, and number to fill a cell.
4. The program checks whether the entered number follows Sudoku rules.
5. If valid, the number is placed in the grid.
6. If invalid, a “Wrong number” message is displayed.
7. The process continues until all empty cells are filled.
8. When the Sudoku is completed, a congratulatory message is shown.
9. The user can enter -1 to generate a new Sudoku puzzle and play again.

C Code

UNDERSTANDING SUDOKU VALIDATION LOGIC

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int board[4][4];

int base[4][4] = {
    {1, 2, 3, 4},
    {3, 4, 1, 2},
    {2, 1, 4, 3},
    {4, 3, 2, 1}};

int isValid(int row, int col, int num)
{
    for (int i = 0; i < 4; i++)
    {
        if (board[row][i] == num)
            return 0;
        if (board[i][col] == num)
            return 0;
    }
    return 1;
}

void generateSudoku()
{
    int temp[4][4];

    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            temp[i][j] = base[i][j];

    for (int i = 0; i < 4; i++)
    {
        int r = rand() % 4;
        for (int j = 0; j < 4; j++)
        {
            int t = temp[i][j];
            temp[i][j] = temp[r][j];
            temp[r][j] = t;
```

```c
            temp[r][j] = t;
        }
    }

    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            board[i][j] = temp[i][j];

    for (int k = 0; k < 6; k++)
    {
        int r = rand() % 4;
        int c = rand() % 4;
        board[r][c] = 0;
    }
}

int main()
{
    int r, c, n;
    int complete;

    srand(time(0));
    generateSudoku();

    printf("4x4 Sudoku Game\n");
    printf("Enter row and column from 1 to 4.\n");
    printf("Enter -1 to generate a new Sudoku.\n");

    while (1)
    {
        printf("\nBoard:\n");
        for (int i = 0; i < 4; i++)
        {
            for (int j = 0; j < 4; j++)
            {
                if (board[i][j] == 0)
```

```c
                printf("%d ", board[i][j]);
            }
            printf("\n");
        }

        complete = 1;
        for (int i = 0; i < 4; i++)
            for (int j = 0; j < 4; j++)
                if (board[i][j] == 0)
                    complete = 0;

        if (complete)
        {
            printf("\n Congratulations! You solved the Sudoku!\n");
            printf("Type -1 to play again.\n");
        }

        printf("\nEnter row col number (1-4) or -1: ");
        scanf("%d", &r);

        if (r == -1)
        {
            generateSudoku();
            continue;
        }

        scanf("%d %d", &c, &n);

        r--;
        c--;

        if (r >= 0 && r < 4 && c >= 0 && c < 4 && board[r][c] == 0 && n >= 1 && n <= 4)
        {
            if (isValid(r, c, n))
            {
                board[r][c] = n;
            }
            else
            {
                printf("Wrong number! Violates Sudoku rules.\n");
            }
        }
        else
        {
            printf("Invalid move!\n");
        }
    }

    return 0;
}
```

Program Output Examples and Messages

GAME RESULTS OVERVIEW

The game effectively displays the **initial Sudoku grid**, alerts for invalid input, and celebrates success upon completion for an engaging experience.

```
4x4 Sudoku Game
Enter row and column from 1 to 4.
Enter -1 to generate a new Sudoku.

Board:
3 . . .
. 3 2 1
1 . 3 4
2 1 . 3

Enter row col number (1-4) or -1: 2 3 5
Invalid move!

Board:
3 . . .
. 3 2 1
1 . 3 4
2 1 . 3

Enter row col number (1-4) or -1: 2 1 4

Board:
3 . . .
4 3 2 1
1 . 3 4
2 1 . 3

Enter row col number (1-4) or -1:
```

What We Learned...

FROM LOGIC TO CODE, AND FROM CODE TO SOLUTION THIS PROJECT SHAPED OUR THINKING.

- Arrays and 2D matrices in C
- Loops and conditional statements
- Functions and modular programming
- Use of random numbers in C programming
- Input validation and logical thinking
- Problem-solving skills
- Team collaboration and project planning

Thank You!